

DynamicAnalyser — Analysis Overview

Static analysis · CI/CD pipeline · Application logs

Correlation & explanation

(slide deck — edit export settings in Marp)

Agenda

1. **What we analyse** — three complementary lenses
2. **Static analysis** — code-first, architecture & risk
3. **CI/CD analysis** — pipeline-first, time & workflow
4. **Log file analysis** — runtime-first, functions & timings
5. **Correlation** — how all three meet at **source code**
6. **Outcome** — ranked fixes, links to GitHub, confidence

Three pillars — same repo, different signals

Lens	Primary input	Question answered
Static	Source tree (AST chunks)	Where is complexity / risk in the code ?
CI/CD	GitHub Actions runs & logs	Which steps waste time in the pipeline ?
App logs	Uploaded traces (tshark, JSON, ...)	Which functions dominate runtime ?

Together they answer: **slow *where* — build, runtime, or implementation?***

Static analysis — what we did

- **Scope:** Repository resolved via GitHub (owner/repo or URL); optional domain/file caps for huge trees (e.g. Wireshark-scale).
- **Mechanism:** Chunk relevant files → **tree-sitter AST** signals → **Layer 1** structural hints → **Claude** for deeper commentary per chunk/domain.
- **Output:** Multi-domain report — hotspots, anti-patterns, maintainability / complexity themes (**not** Wall-clock “this CI step took 9 min”).
- **Best for:** Architecture review, risky modules, onboarding **before** or **alongside** runtime/CI data.

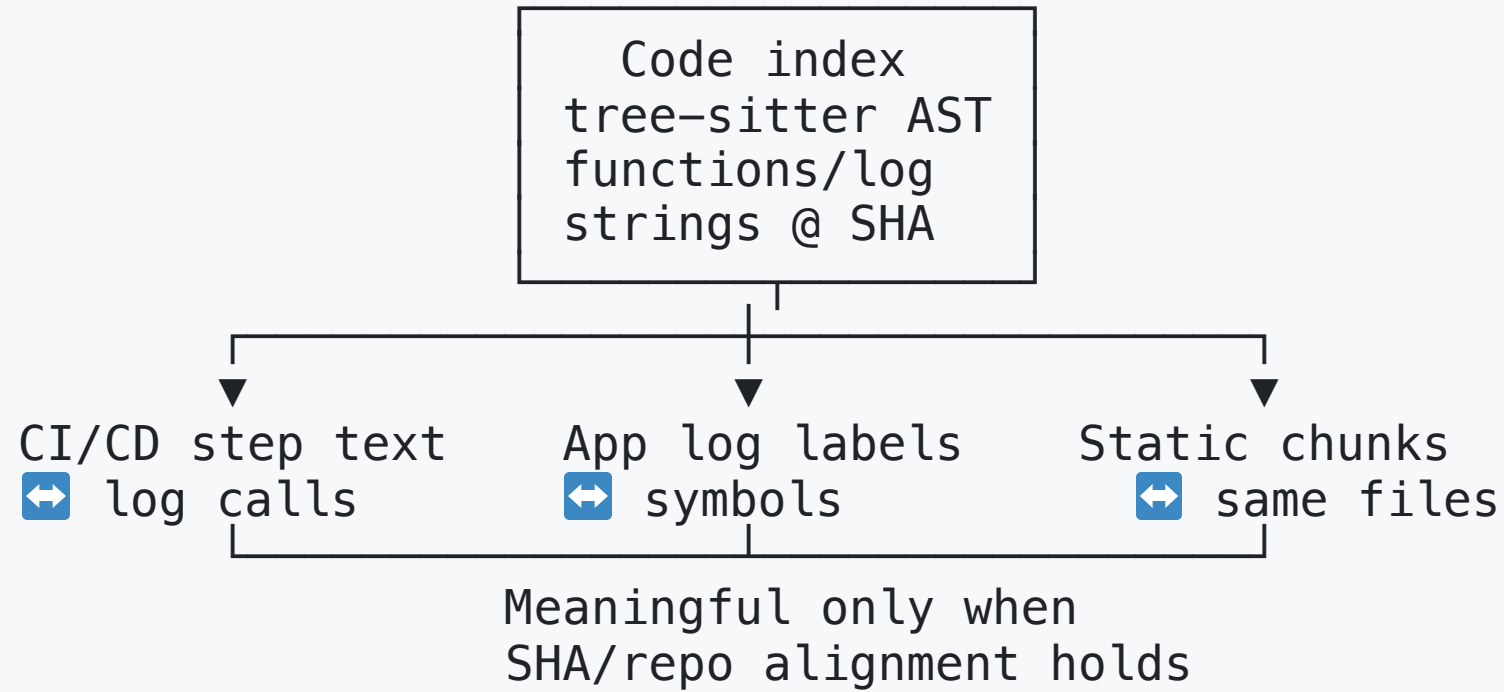
CI/CD analysis — what we did

- **Ingestion:** Track repo → fetch **workflow runs** → download **step logs** → parse into **StepTiming** rows (duration, excerpts).
- **Ranking:** **Bottleneck ranker** — statistical composite (duration share, anomaly vs history).
- **Indexing:** **AST index** at a commit — functions + optional **log string** → **line** map from CI-facing logging calls.
- **Correlation:** **Trace correlator** matches step text to **indexed log lines / symbols** (exact → fuzzy → fallback).
- **AI:** **Root cause**, suggestions (cache, parallelism, ...), **estimated savings** — grounded in **timings + snippets**, estimates are **directional**.

Application log analysis — what we did

- **Upload:** Plain-text logs — **auto format detection** or explicit format (**tshark**, JSON lines, syslog, **heuristic**, custom regex).
- **Parse:** Extract **function-name + duration** records (`UniversalLogRecord` → DB).
- **Indexing:** Same **CodeIndex** as CI/CD — **GitHub tree** or **local clone** (`AST_INDEX_LOCAL_ROOT` + path) for speed on huge repos.
- **Correlation:** Log labels → **IndexedFunction** (**exact / normalised / fuzzy / token overlap**) → **file:line** + optional callers.
- **Links:** GitHub **blob URLs** use indexed **commit SHA** so links stay valid (**not** hardcoded `main` vs `master`).
- **AI:** Bottleneck & refactor hints scoped to **slow calls** (labels may be synthetic vs real symbols — **interpret accordingly**).

Correlation — the shared spine



One index · multiple consumers · **commit-aware** links

CI/CD ↔ code — how it connects

- Steps expose **log excerpts**; indexer finds `logging.info("...")` -style strings or symbol names in code.
- **Trace view**: Step → **source file / line** → optional **call chain** via reverse call graph.
- **Gap**: If CI logs never contain strings we indexed, match degrades to **fuzzy / grep** — confidence drops.

App logs ↔ code — how it connects

- Log lines carry names like `tcp_retransmission_analysis` or `dissect_*` / `gtpv2` (after conversion from PCAP fields).
- Matcher maps to `IndexedFunction.function_name` — **exact** rare for synthetic labels → **token overlap** / fuzzy for Wireshark-scale indexes.
- **Result:** Table of **log label** → **best matching symbol** → **path:L** — click-through to GitHub at indexed SHA.

Static ↔ CI/CD & logs — how it relates

- **Static** highlights **modules** worth watching; **CI** shows which jobs touch those paths (indirectly, via team knowledge).
- **Logs** prove **which symbols** burn time at runtime — cross-check with **static** “heavy” domains.
- **No automatic merge** of three scores into one number — **human triage**: pipeline vs runtime vs refactorability.

Data & limits — be transparent on slides

Topic	Caveat
LLM savings	Minutes saved are estimates , not A/B measured.
Synthetic logs	Dummy / PCAP-derived labels $\neq$ verified Wireshark internals.
Huge uploads	Preview uses first slice of file; raise <code>APP_LOG_MAX_SIZE_MB</code> if needed.
PCAP	Raw packets $\neq$ <code>elapsed=</code> — convert with <code>tshark -T fields</code> + script (see <code>scripts/pcap_tsv_to_app_log.py</code>).

What we demonstrated (your sessions)

- **Wireshark CI/CD:** Slow steps → suggestions (**ccache**, MSYS2 cache, **parallel CTest**) — plausible **themes**, validate in YAML.
- **Wireshark app logs:** Index local clone → **partial correlation** (NAS/GTP-style labels  **packet-*.c**).
- **dummy.log:** Synthetic timings → AI picks slowest label — **illustrative**, not proof of real code paths.

Takeaways for stakeholders

1. **Three analyses** — complementary; **correlation** ties them to **indexed source**.
2. **Trust** — timing & paths **measured/indexed**; **LLM narrative & savings** — **review**.
3. **Next** — tighten workflows from CI suggestions; align log labels with symbols for **better match rate**.

Thank you / Q&A

DynamicAnalyser — Static · CI/CD · Logs · Correlation

Export this file with **Marp** → PDF or PowerPoint for presentation.